

Market Basket Analysis

Finding frequent itemsets in the IMDB Movies Dataset

Greta Zehnder

June 2026

Contents

1	Introduction	2
2	Dataset and preprocessing	2
2.1	Dataset	2
2.2	Preprocessing	2
3	Exploratory analysis and support threshold selection	3
3.1	Actor frequency analysis	3
3.2	Support threshold selection	3
4	Frequent itemset mining with Apriori	3
4.1	Theoretical background	3
4.2	Implementation	4
5	Results	4
6	Scalability experiment	5
6.1	Experimental setup	5
6.2	Results and discussion	5
7	Conclusions	6
	Declaration of Academic Integrity	9

1 Introduction

The market-basket model, described in [1], is one of the most widely used frameworks for discovering patterns in transactional data. The idea is simple: we have a set of *items* and a collection of *baskets*, each containing a small subset of those items, and we want to find which groups of items tend to appear together in many baskets. More formally, the *support* of an itemset I is the number of baskets that contain all items in I , and I is called *frequent* if its support is at least some threshold s [1].

The original motivation came from retail (a supermarket chain wanting to know which products customers buy together) but the same approach applies to any domain where data can be cast as a collection of sets. As discussed in [1], examples range from detecting plagiarism between documents to identifying biomarkers associated with diseases.

In this project, the model is applied to cinema: each film in the IMDB Top 1000 dataset is treated as a basket, and its four main actors are the items. The question we ask is which actors appear together across multiple films often enough to qualify as a frequent pattern. To answer it, a custom implementation of the Apriori algorithm [1] is built from scratch and run on the full dataset of 1000 films. The report also includes a small scalability experiment to observe how the algorithm’s runtime changes as the number of transactions grows.

The rest of the report is organized as follows. Section 2 describes the dataset and how it is prepared. Section 3 covers the exploratory analysis used to choose the support threshold. Section 4 explains the Apriori algorithm and its implementation. Section 5 presents the frequent itemsets found, and Section 6 reports the scalability experiment.

2 Dataset and preprocessing

2.1 Dataset

The dataset used is the *IMDB Top 1000 Movies and TV Shows Dataset* [2], freely available on Kaggle. It contains 1000 entries, each representing a highly rated title on IMDB, with 16 columns covering information such as title, release year, genre, rating, director, and gross revenue. For this project, the only columns that matter are the four actor fields: `Star1`, `Star2`, `Star3`, `Star4`, which list the main cast members of each film.

The mapping to the market-basket setting is immediate: each row is a basket and the four actors are its items. No structural transformation is needed beyond selecting the relevant columns.

2.2 Preprocessing

The preprocessing step is minimal. After selecting the four actor columns, rows with any missing values are dropped via `dropna()`. In practice, all 1000 rows survive this step, meaning the actor fields are fully populated throughout the dataset. Each row is then converted to a plain list of four names, giving 1000 transactions ready to be passed to the algorithm.

The support threshold used to decide which actors are considered frequent is determined in the next section and enforced internally by Apriori at the first pass; the full set of 1000

transactions is passed to the algorithm without any prior filtering.

3 Exploratory analysis and support threshold selection

3.1 Actor frequency analysis

Before running Apriori, it is useful to look at how often each actor appears across the 1000 films. This serves two purposes: getting a sense of which actors are prominent in the dataset, and deciding on a reasonable minimum support threshold.

The dataset contains 2709 distinct actors in total, which is a large and sparse item space relative to 1000 baskets of only four items each. Table 1 shows the ten most frequent actors. Robert De Niro leads with 17 appearances, followed by Tom Hanks with 14 and Al Pacino with 13.

Actor	Appearances
Robert De Niro	17
Tom Hanks	14
Al Pacino	13
Brad Pitt	12
Clint Eastwood	12
Christian Bale	11
Leonardo DiCaprio	11
Matt Damon	11
James Stewart	10
Michael Caine	9

Table 1: Top 10 most frequent actors in the dataset.

3.2 Support threshold selection

The first threshold tried was 1%, corresponding to a minimum count of 10 out of 1000 films. This turned out to be too strict: only 9 actors clear the bar, leaving almost nothing to mine at higher itemset sizes. Lowering the threshold to 0.5% (a minimum count of $\lfloor 0.005 \times 1000 \rfloor = 5$) brings the number of frequent actors up to 79, which is much more workable. This is the threshold used for all subsequent analysis. The 1000 transactions are passed to Apriori as-is; the algorithm itself discards infrequent items at the first pass, exploiting monotonicity to avoid counting combinations that cannot possibly be frequent.

4 Frequent itemset mining with Apriori

4.1 Theoretical background

The Apriori algorithm [1] finds frequent itemsets by exploiting a property called *monotonicity*: if an itemset I is frequent, then every subset of I must also be frequent [1]. Equivalently, if an itemset fails to meet the support threshold, none of its supersets can

be frequent either. This lets the algorithm avoid counting the vast majority of possible combinations, working instead level by level and only considering candidates whose smaller subsets have already been confirmed as frequent.

The algorithm makes one pass over the data for each itemset size k . At each level, two collections are maintained [1]:

- C_k : the *candidate* k -itemsets that need to be counted;
- L_k : the *frequent* k -itemsets, i.e., those in C_k that meet the threshold.

Starting from C_1 (all individual items), the first pass produces L_1 . For each subsequent size k , candidates in C_k are generated by merging pairs of itemsets from L_{k-1} and keeping only those all of whose $(k-1)$ -subsets are in L_{k-1} ; the data is then scanned to count them and filter out those below the threshold. The process stops as soon as a level yields no frequent itemsets [1].

As noted in [1], the most memory-intensive step is typically the counting of pairs, since the number of possible pairs grows quadratically with the number of items. Monotonicity-based pruning is what makes this manageable: by the time pairs are counted, only the $|L_1|$ frequent singletons remain in play.

4.2 Implementation

The implementation consists of three helper functions and a main driver. `count_singletons(baskets)` performs the first pass, scanning all baskets and returning a `Counter` of item frequencies, which directly produces L_1 . `generate_candidates(prev_frequent, k)` takes the itemsets in L_{k-1} and builds C_k by merging pairs and pruning any candidate whose $(k-1)$ -subsets are not all in L_{k-1} , implementing the monotonicity check from [1]. `count_candidates(baskets, candidates, frequent_items, k)` scans the baskets to count each candidate in C_k , restricting attention within each basket to items already in L_1 : a second application of monotonicity that avoids generating combinations involving singletons that were already pruned at the first pass.

The main `apriori(baskets, min_count)` function calls these helpers in sequence for $k = 1, 2, 3$, stops early if C_k is empty, and returns a dictionary from k to the corresponding L_k with counts. It is run on the full 1000 transactions with `min_count = 5`.

5 Results

Running Apriori on the 1000 transactions with `min_count = 5` produces 79 frequent singletons, 3 frequent pairs, and 1 frequent triple. Support values are expressed as fractions of the 1000 original films.

The three frequent pairs are shown in Table 2. The most common co-acting duo is Daniel Radcliffe and Rupert Grint, who appear together in 6 films. The other two pairs (Radcliffe with Emma Watson, and Watson with Grint) each appear in 5. All three pairs are from the Harry Potter franchise, reflecting how consistently the same core cast appears across the series.

Actor pair	Support	Count
Daniel Radcliffe, Rupert Grint	0.0060	6
Daniel Radcliffe, Emma Watson	0.0050	5
Emma Watson, Rupert Grint	0.0050	5

Table 2: Frequent actor pairs found by Apriori.

Moving to triples, the only frequent 3-itemset is the full Harry Potter core trio (Radcliffe, Watson, and Grint together) with a count of 5 (Table 3).

Actor triple	Support	Count
Daniel Radcliffe, Emma Watson, Rupert Grint	0.0050	5

Table 3: Frequent 3-itemset found by Apriori.

The fact that all discovered patterns involve the same franchise is not surprising. The IMDB Top 1000 over-represents popular series, and with a minimum support of 5 films the only actors who can form frequent pairs are those who starred in multiple entries of the same franchise. The Harry Potter series, with 8 mainline films, is the natural candidate.

6 Scalability experiment

6.1 Experimental setup

To get a sense of how the algorithm’s runtime scales with the amount of data, the experiment runs Apriori on three subsets of increasing size: 30%, 60%, and 100% of the 1000 transactions (i.e., 300, 600, and 1000 baskets). Each configuration is repeated 10 times, with runtimes measured using `time.perf_counter()` for sub-millisecond precision, and the average is reported. A fixed random seed (42) ensures the same subsamples are drawn each time. The minimum count threshold is kept fixed at 5 across all sizes, so that differences in runtime reflect only the effect of dataset size and not a change in how many itemsets are considered frequent.

6.2 Results and discussion

The results are summarized in Table 4 and visualized in Figure 1. Going from 300 to 1000 baskets (roughly a $3.3\times$ increase) the average runtime grows from 0.0006 s to 0.0095 s, a factor of about 16. This superlinear growth is the expected behavior for Apriori.

Fraction	Baskets	Avg runtime (s)
0.3	300	0.0006
0.6	600	0.0026
1.0	1000	0.0095

Table 4: Apriori average runtime as dataset size increases (10 runs each).

The reason the growth is superlinear comes from an interaction between dataset size and the fixed threshold. At 300 baskets, a minimum count of 5 corresponds to an effective support of about 1.7%, which is much stricter than the 0.5% intended for the full dataset. As a result, almost no pairs or triples reach the threshold and the algorithm finishes very quickly, not because it is fast in any meaningful sense, but because it has almost nothing to count. As the number of baskets increases, more itemsets gradually clear the threshold, the candidate set grows, and the counting step becomes progressively more expensive. This explains why the runtime jumps so sharply between the 60% and 100% configurations.

In absolute terms, the runtimes are tiny throughout, the entire experiment runs in milliseconds. This is a consequence of the dataset being small and the number of frequent actors being limited. On a larger and denser dataset, with thousands of items and lower support thresholds generating far more candidates, the same superlinear trend would translate into much more significant runtime differences.

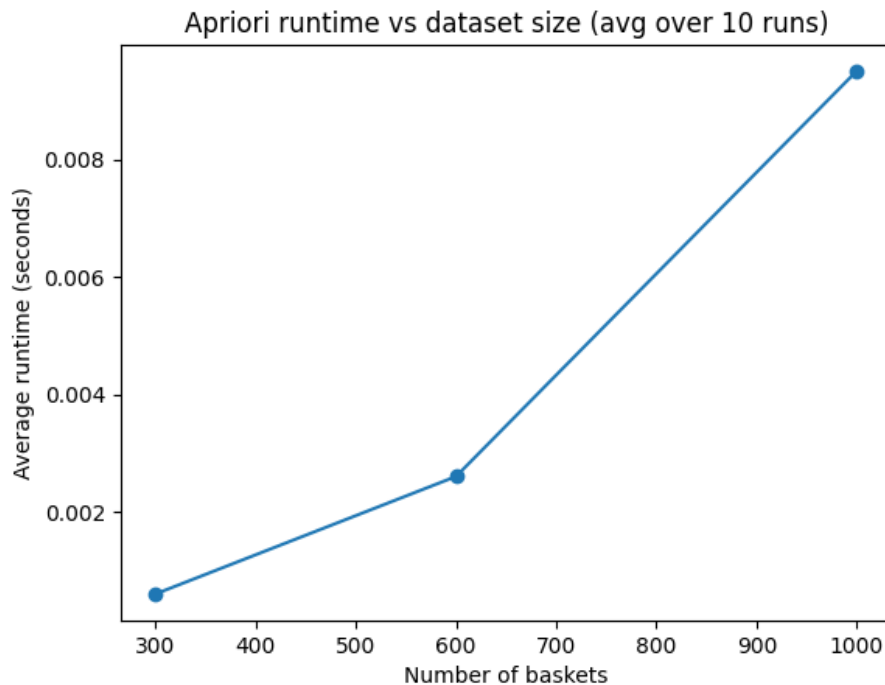


Figure 1: Apriori runtime vs dataset size (average over 10 runs).

7 Conclusions

The goal of this project was to apply market basket analysis to a movie dataset, treating each film as a basket and its four main actors as items, and to discover which actor combinations appear frequently enough across the IMDB Top 1000 to be considered a recurring pattern.

A minimum support threshold of 0.5% (a count of at least 5 out of 1000 films, as previously shown) was selected after finding that a stricter 1% threshold left only 9 frequent actors, too few to produce any interesting co-occurrence patterns. A custom Apriori implementation was then run on the full dataset, yielding 79 frequent singletons, 3 frequent

pairs, and 1 frequent triple. All three patterns involve the same actors: Daniel Radcliffe, Emma Watson, and Rupert Grint, the core cast of the Harry Potter series. This outcome makes intuitive sense, since with 8 mainline films in the franchise, the same trio recurs often enough to clear the threshold, while actors who appeared in multiple acclaimed but standalone films do not.

The scalability experiment confirmed that Apriori’s runtime grows superlinearly with the number of transactions. Tripling the dataset from 300 to 1000 baskets caused a roughly 16-fold increase in average runtime, driven by the growing number of candidates that accumulate enough support to be counted as the data grows. The absolute runtimes remain negligible on this dataset, but the trend points clearly to the challenges that would arise on larger, denser collections.

A natural next step would be to run the same analysis on a larger and more diverse movie database, where the support threshold could be lowered enough to surface patterns beyond franchise recurrence. Building association rules on top of the frequent itemsets (to move from “these actors often appear together” to “if actor A is in a film, actor B is likely to be too”) would also be a meaningful extension of what has been done here.

References

- [1] J. Leskovec, A. Rajaraman, and J. D. Ullman, *Mining of Massive Datasets*, 3rd ed., Cambridge University Press, 2020. Chapter 6: Frequent Itemsets. <http://www.mmds.org>
- [2] H. Shankhdhar, *IMDB Top 1000 Movies and TV Shows Dataset*, Kaggle, 2021. <https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows>

Declaration of Academic Integrity

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.